



Vietnam National University of HCMC
International University
School of Computer Science and Engineering



Object Oriented Programming

Computer Program & Java

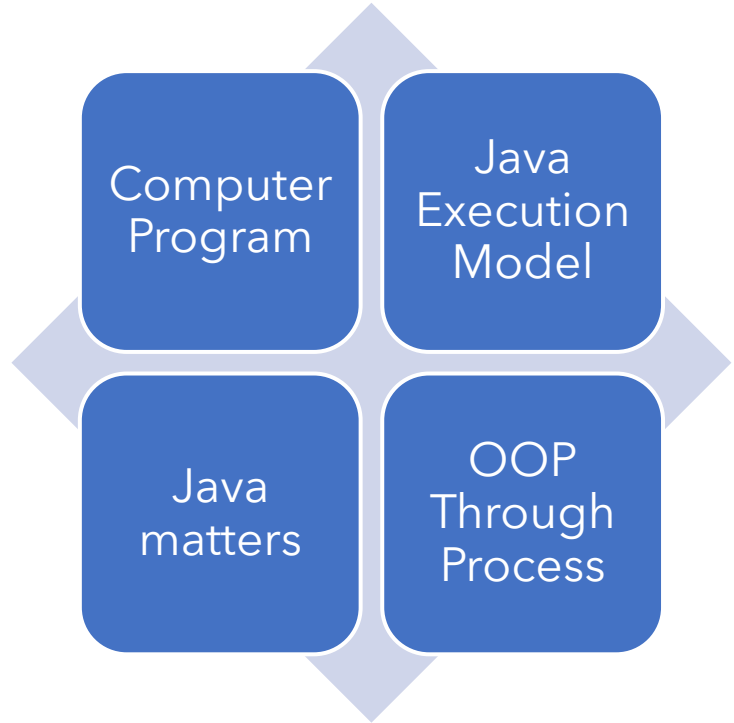
Application

Instructor: Le Thi Ngoc Hanh, Ph.D

ltnhanh@hcmiu.edu.vn

Semester – Spring 2026

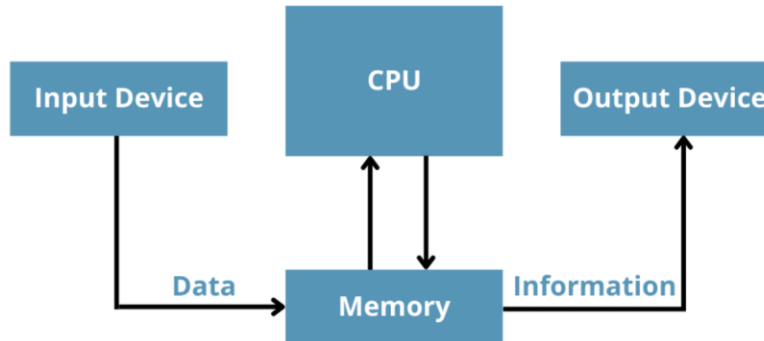
Outline



What is a Computer?

A Computer Is a Deterministic Machine

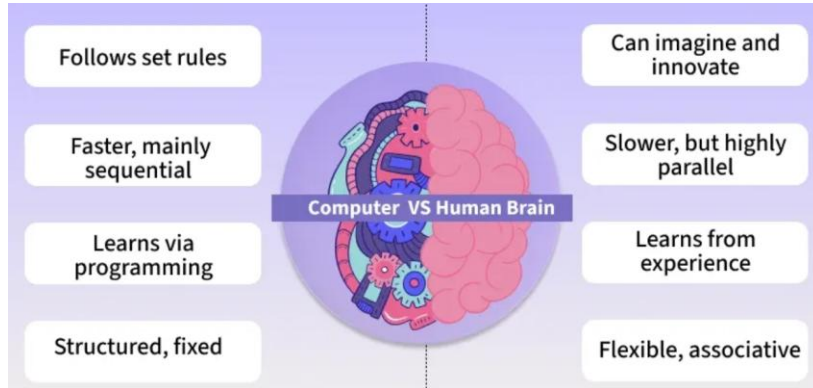
- ☾ CPU: executes instructions
- ☾ Memory: stores data & programs
- ☾ Input / Output: interaction with the world



Computer Program?

Computer Programming:

- ✧ Writing instructions for a computer
- ✧ Instructions must be: Precise, Unambiguous, Ordered

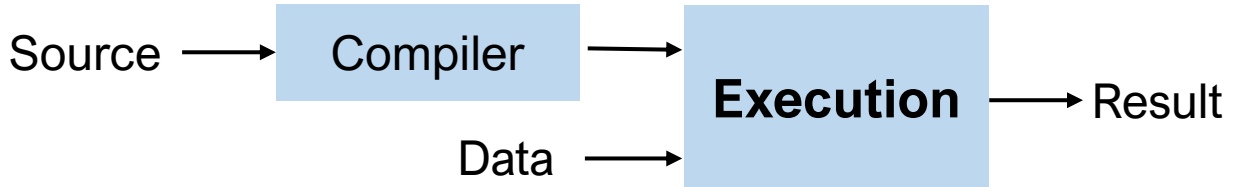


What Is a Programming Language

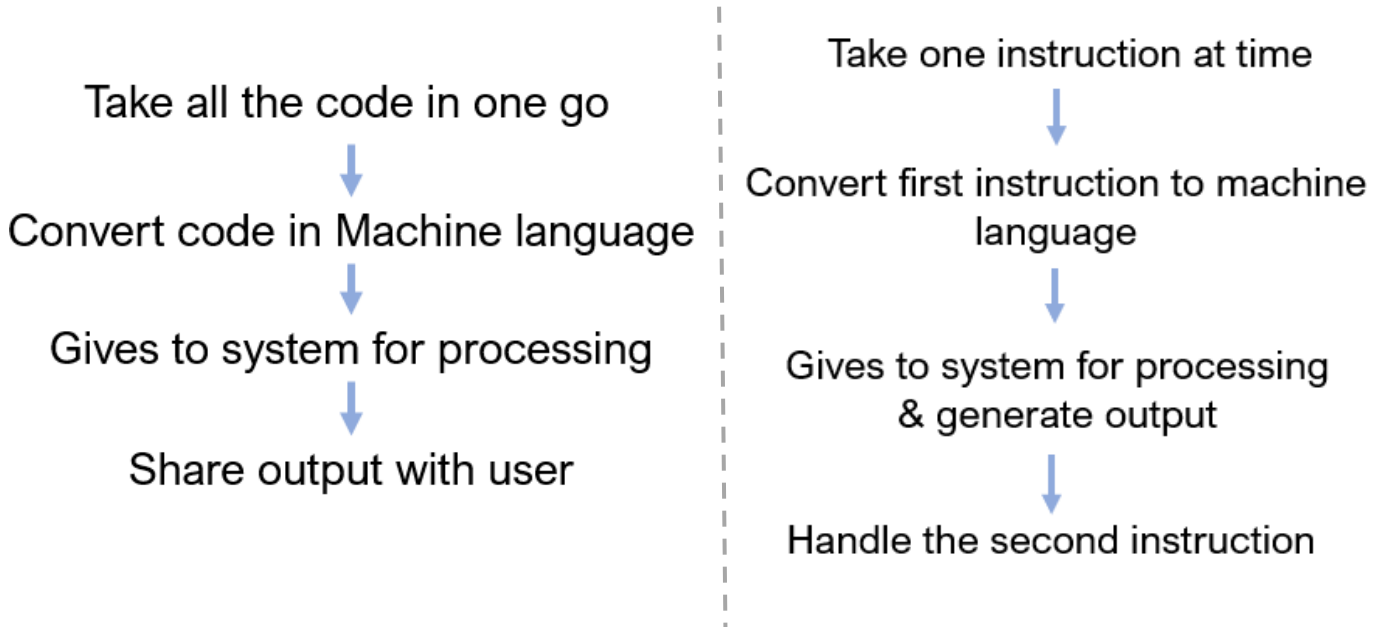
Programming Languages:

- ☾ Bridge between humans and machines
- ☾ Levels:
 - Low-level (close to hardware)
 - High-level (human-friendly)

How Programs Are Executed



Compiler vs Interpreter

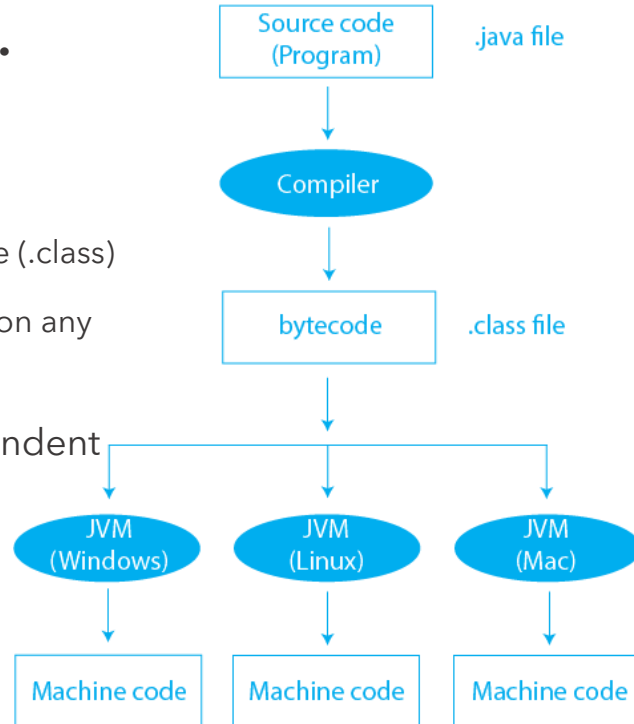


Java's Execution Model

Java uses a hybrid execution model.

Steps in Java Compilation:

- Source code (.java)
 - Compiler (javac): converts .java files to Bytecode (.class)
 - JVM (Java Virtual Machine): Executes Bytecode on any platform.
- Bytecode: Intermediate, platform-independent format.
- JVM: Converts Bytecode into machine code specific to the host system.



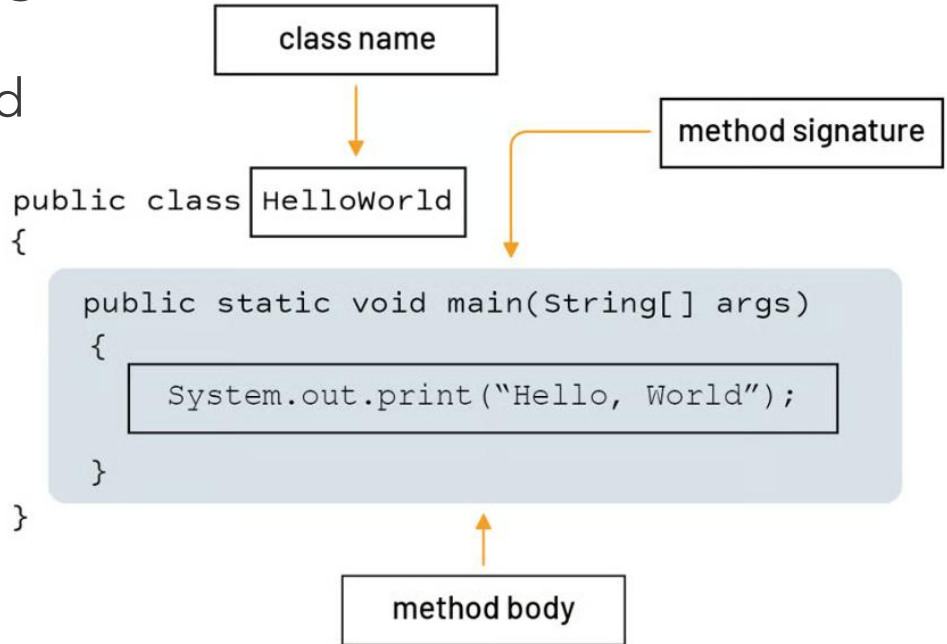
Why Java for OOP?

- ☾ Java is designed to strongly support Object-Oriented Programming
- ☾ Java was created with OOP as its core paradigm, not as an add-on. Its language design and runtime environment actively encourage good OOP practices.

Structure of a Java Program

Basic Java Program:

- Class
- main method
- Entry point



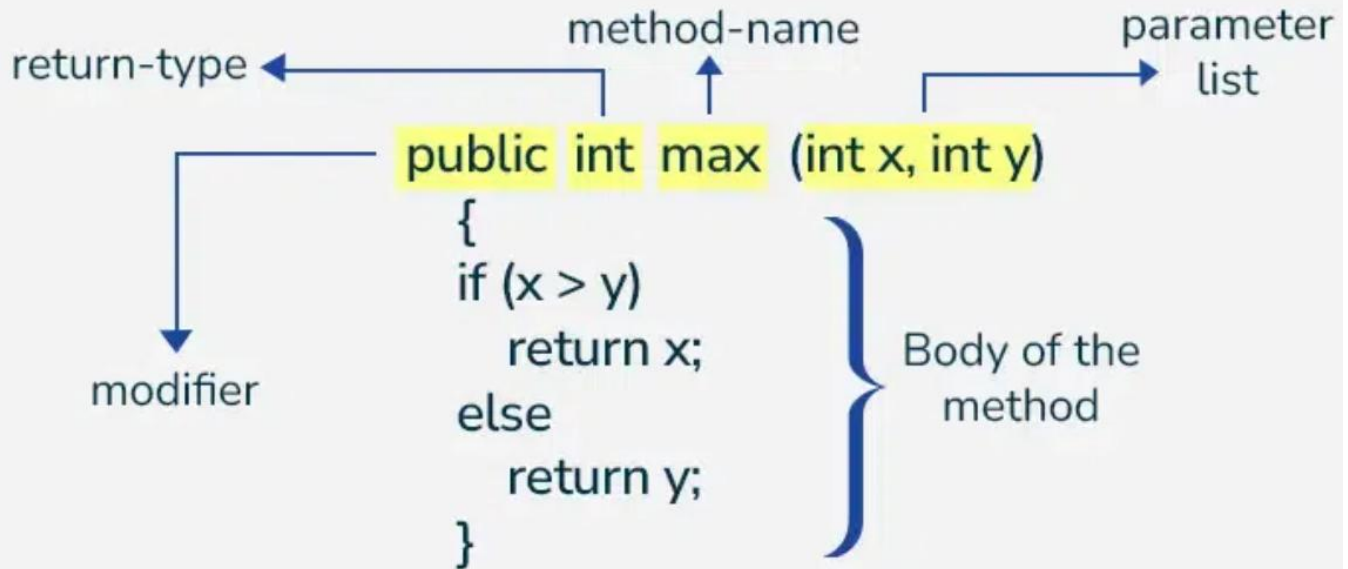
More example

```
// File: Main.java
public class Main {
    public static void main(String[] args) {
        Student s = new Student();
    }
}

class Student {
}

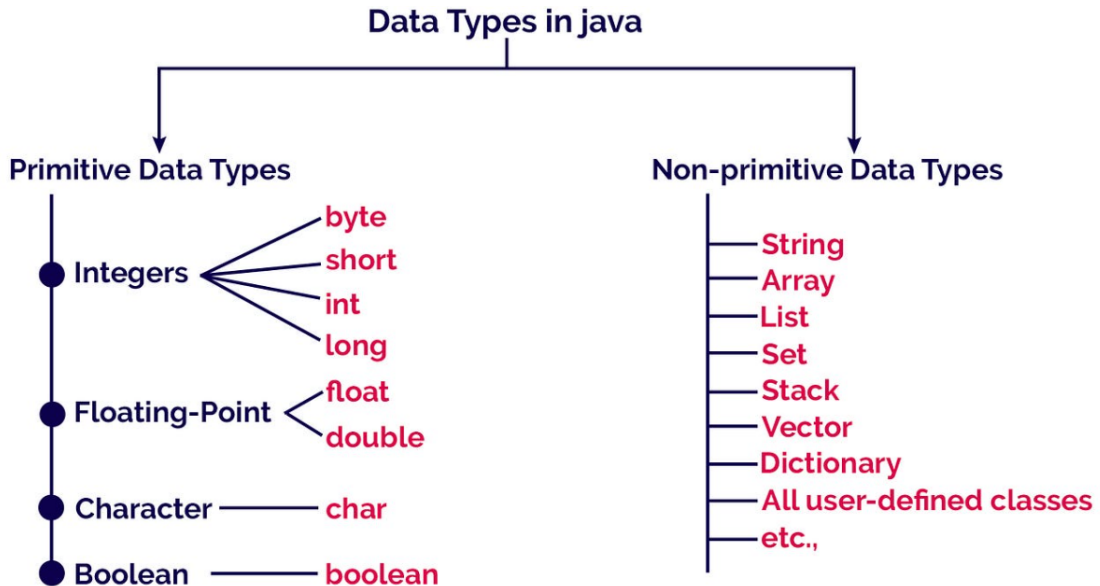
class Teacher {
}
```

Simple Java Program



Java Data Types

Java is a **strongly typed** language: Every variable in Java must have: a type, a name, a value.



Pass-by-Value vs Pass-by-Reference

★ Pass-by-Value (Reference Types / Objects)

- ★ Java always passes arguments by value
 - For objects: the value passed is a copy of the reference
 - Both variables refer to the same object in memory
 - Modifying the object's state inside a method: Affects the original object
 - Reassigning the parameter: Does NOT affect the caller's reference

```
void changeValue(int num) {  
    num = 10;    // No effect outside the method  
}
```

```
void modifyObject(MyObject obj) {  
    obj.value = 20;    // Changes persist outside the method  
}
```

More Example

```
public class PassingExample {  
  
    public static void main(String[] args) {  
        int x = 5;                // x is initialized to 5  
        changex(x);               // pass a copy of x to the method  
        System.out.println(x);    // still prints 5  
    }  
  
    public static void changex(int x) {  
        x++;                      // modifies only the local copy  
        System.out.println(x);    // prints 6  
    }  
}
```

More Example

```
public class PassObjectExample {  
  
    public static void main(String[] args) {  
  
        Point p1 = new Point();    // create a Point object  
        p1.x = 1;  
        p1.y = 2;  
  
        System.out.println("Before change: x = " + p1.x + ", y = " + p1.y);  
  
        change(p1);                // pass object reference (by value)  
  
        System.out.println("After change: x = " + p1.x + ", y = " + p1.y);  
    }  
  
    private static void change(Point p) {  
        p.x = 2;                    // modify object state  
        p.y = 1;  
    }  
}
```

Object Types



Object is the **root (parent) class of all Java classes**



It is defined in the package `java.lang.Object`



Every Java class **implicitly extends Object**



Unless it explicitly extends another class

Example:

```
class Student {  
}
```

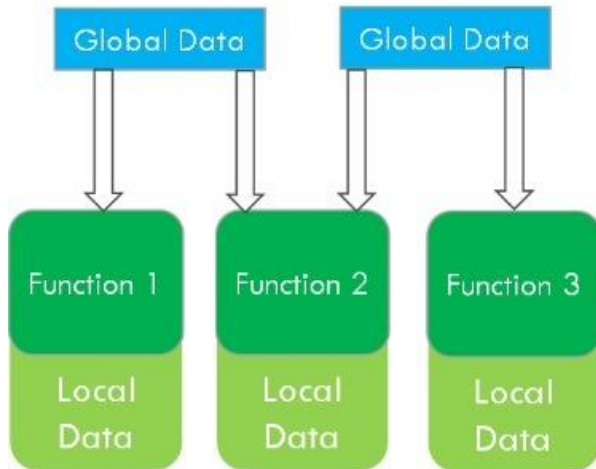
```
class Student extends Object {  
}
```

Object – Oriented Design

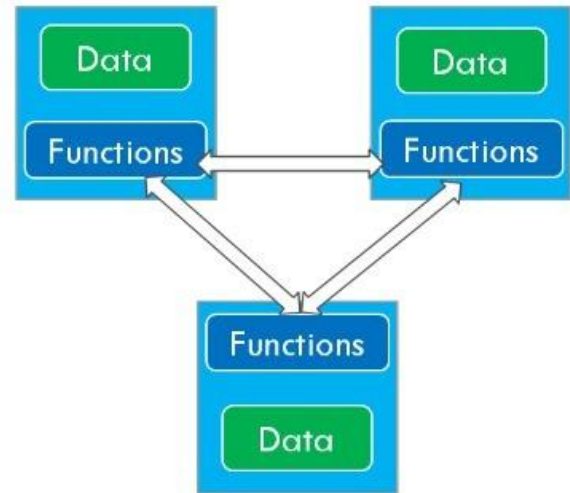
- ☾ A robust object model is not accidental
- ☾ Good OO design starts with problem understanding
- ☾ Design is iterative, not one-shot
- ☾ Early designs are expected to change

Structured vs Object-Oriented Design

Procedural Oriented Programming



Object Oriented Programming



Object-Oriented Thought Process

- ✧ Knowing the difference between the interface and the implementation.
- ✧ Thinking more abstractly.
- ✧ Giving the user the minimal interface possible .

Poor OO design usually violates one or more of them!

Interface vs Implementation

★ Interface

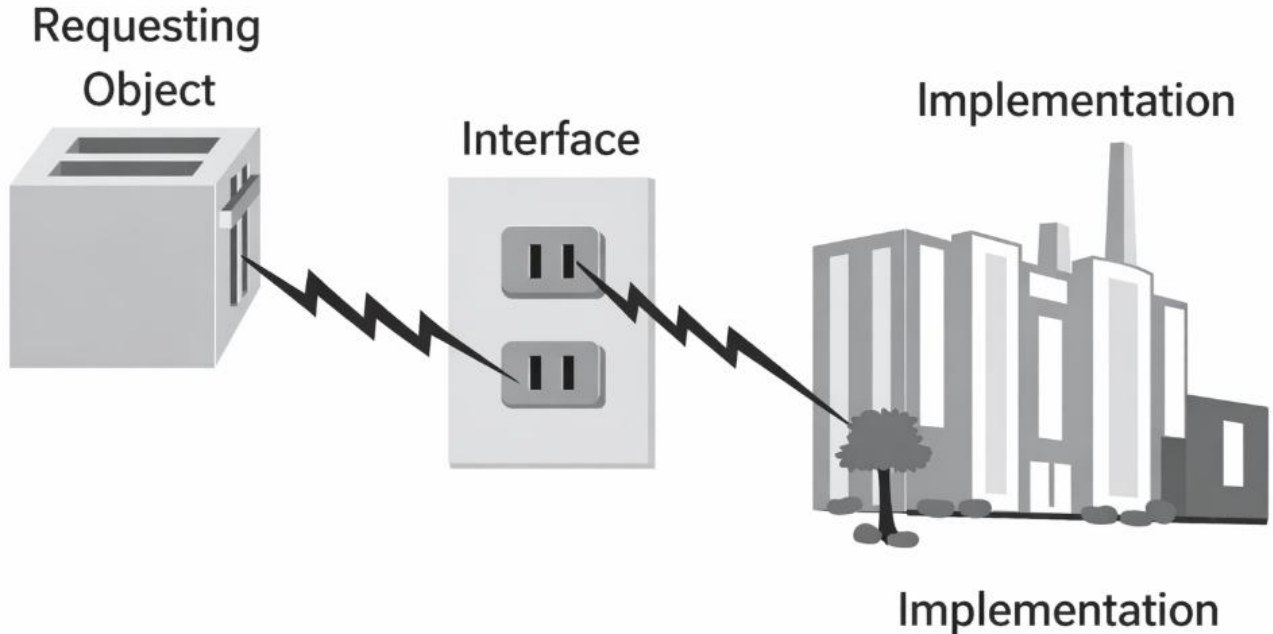
- What the user needs to know
- What services are provided

★ Implementation

- How the service is carried out
- Hidden internal details

★ Users depend on **interfaces**, not implementations

Interface vs Implementation



More Examples

```
// Interface: WHAT the service does
public interface Payment {
    void process(double amount);
}
```

```
// Implementation Sample
public class CreditCardPayment implements Payment {
    @Override
    public void process(double amount) {
        System.out.println("Processing credit card payment: $" + amount);
    }
}
```

Changing [Implementation vs Interface]

☾ Changing implementation:

- Should NOT affect users

☾ Changing interface:

- Likely breaks user code

☾ Interchangeable implementations must behave identically at the interface level

End users vs Programmers

- End users see GUI-users of an application
- Programmers see class interface-users of classes

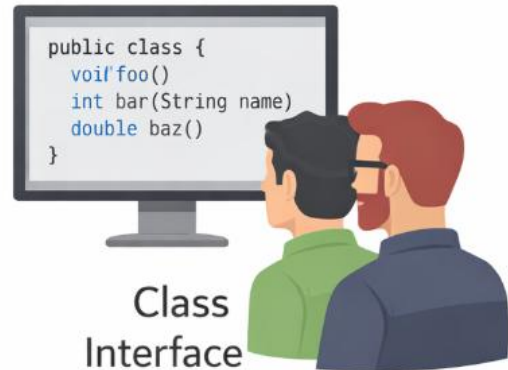
End Users



GUI

Users of an application

Programmers



Class
Interface

Users of classes

The Implementation

- ☾ The interface should expose **services**, not details
- ☾ The interface should **NOT** include: Internal data structures, Helper methods, Algorithmic steps
- ☾ Implementation details must be **hidden from users**

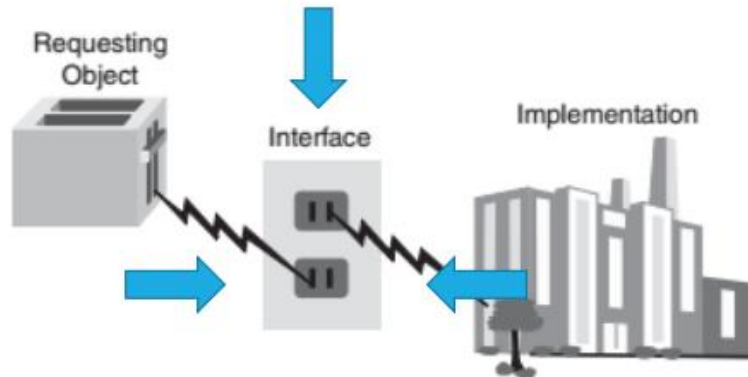
The ultimate goal of the design



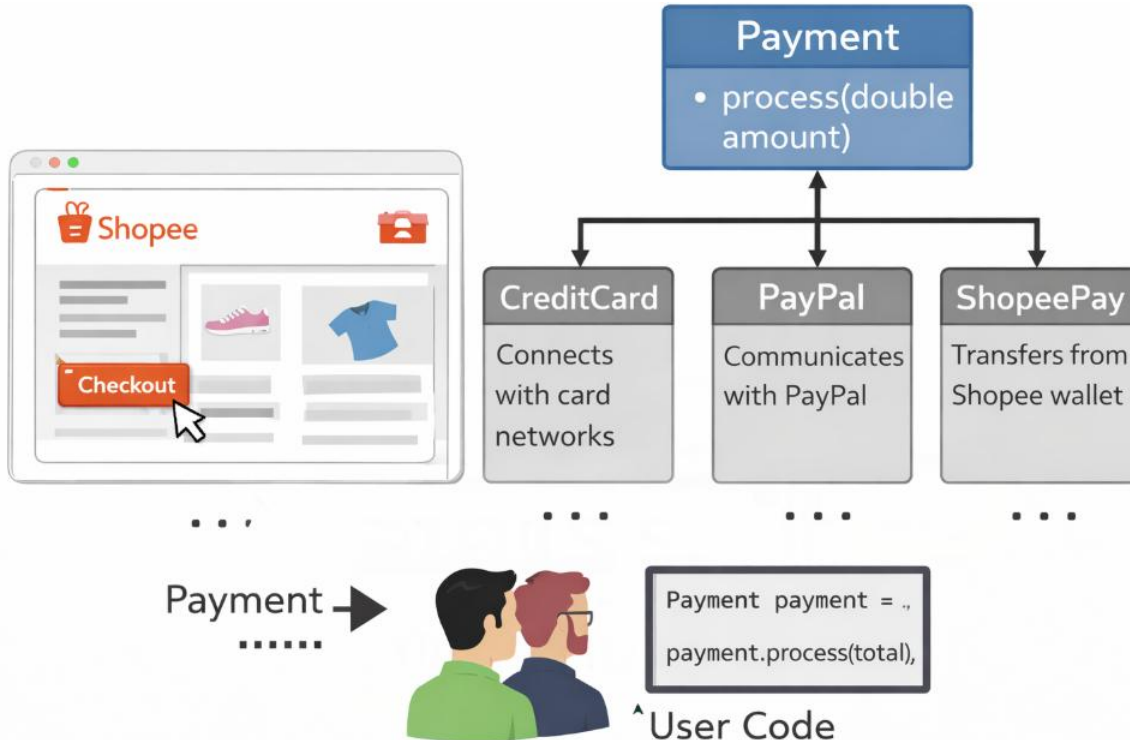
A change to the implementation
should not require
a change to the user's code

The interface specification

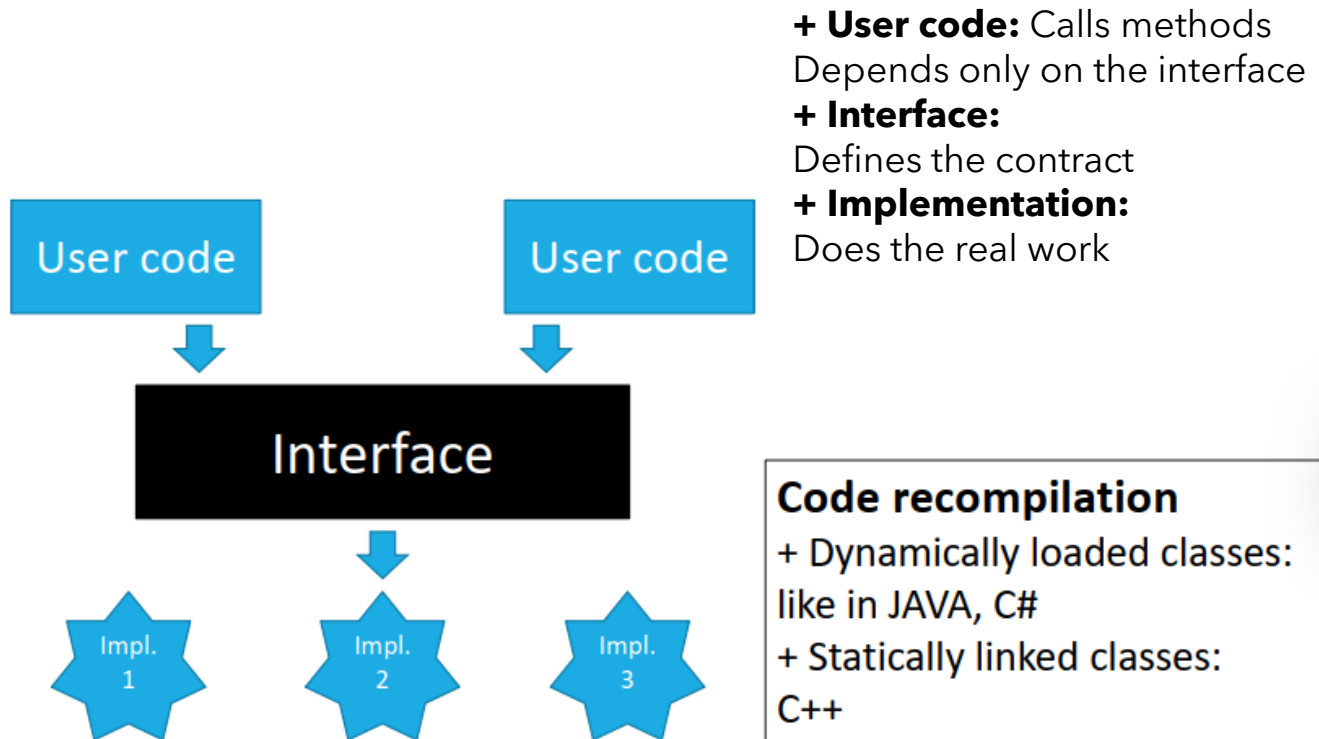
Both the users and the implementation must conform to the interface



Interface/Implementation example



Separation of interfaces from implementation



Usefulness, reusability and levels of abstraction

In OOP, classes can be reused.

Reusable classes tend to have interfaces more abstract than concrete.

- ☾ Highly abstract interface are often more **useful** than highly concrete interface, but not always the case.
- ☾ It is possible to have a very useful.

Design Goal

★ To achieve **abstract, highly reusable classes**:

- Focus on what users need to do
- Avoid exposing how things are done

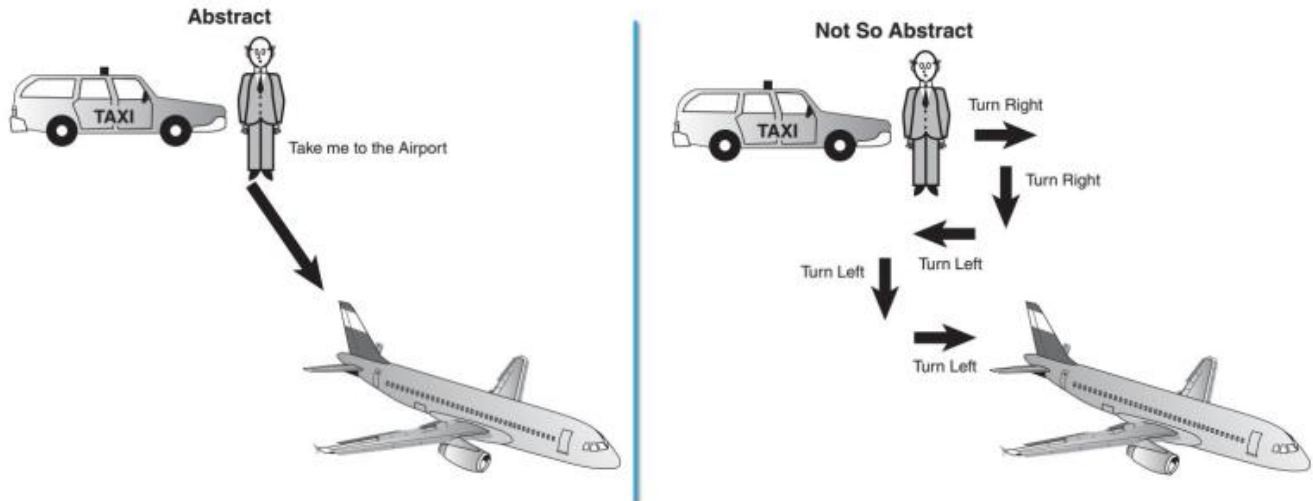
★ User interfaces should:

- Be stable over time
- Change less frequently than implementations

★ A well-designed interface:

- Supports multiple implementations
- Minimizes impact of future changes

Abstract vs concrete interface



Which one is more reusable ?

Minimal interface possible

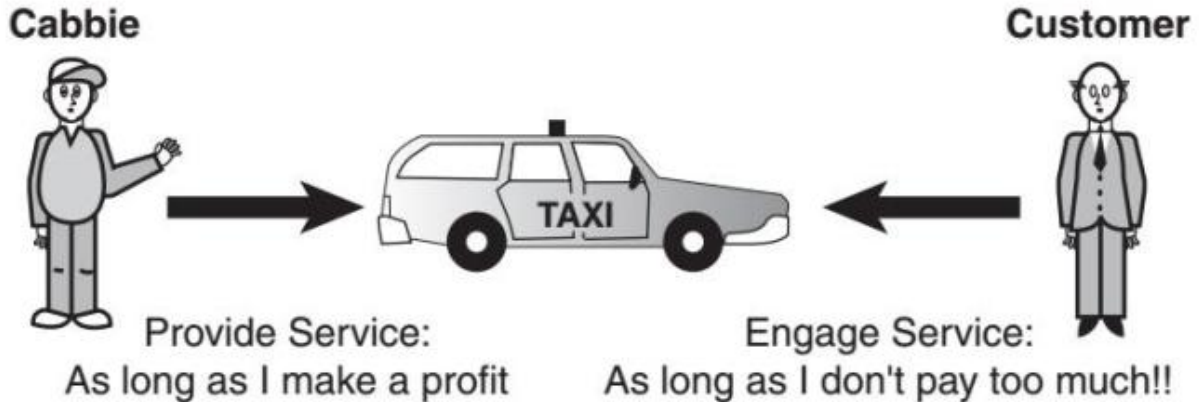
Provide as little knowledge of the inner workings of as possible

- ✧ Give only what users need. Better to have to add than to give more than needed.
- ✧ Starting from private interfaces
- ✧ Design from a user's perspective (not from information systems viewpoint)
- ✧ Design a class from requirements
- ✧ Design with users of the class

Abstraction & Minimal Interface

- ☾ Abstract interfaces are often **more reusable**
- ☾ Describe *what*, not *how*
- ☾ Provide the **minimal interface possible**
- ☾ Better to add later than expose too much early

Determining the users



Next lecture